HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

Autism Spectrum Disorder

Group 6 — Machine Learning

Trần Trung Hiếu - 20235934
Phạm Song Hào - 20235930
Nguyễn Đức Hoàng - 20235938
Vũ Xuân Sơn - 20235995
Đặng Quốc Vượng - 20236015

ONE LOVE. ONE FUTURE.

Outline

Autism Spectrum Disorder and early screening

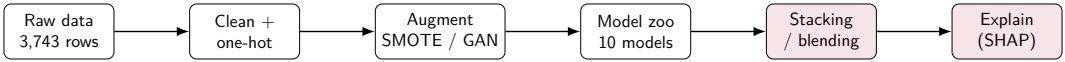
- Autism Spectrum Disorder (ASD) changes how a person communicates, behaves, and interacts with others. Signs usually appear in early childhood.
- A full clinical diagnosis is slow and costly: it needs trained specialists, long interviews, and often months on a waiting list.
- A cheap, fast screening step helps families reach support sooner — when that support helps the most.
- Screening does not replace a doctor. It only flags who should be seen first.

From a questionnaire to a prediction

- AQ-10: ten short yes/no questions about everyday behaviour (`A1`–`A10`). Each autistic-like answer adds 1 point.
- Each person also gives simple facts: age, sex, jaundice at birth, family history of autism, and who filled in the form.
- A single fixed cut-off on the score is unreliable in the grey zone (score 4–6), where ASD and non-ASD cases overlap.

Goal: learn from the AQ-10 answers plus the background facts to flag likely ASD cases accurately and honestly, with no data leakage.

Project pipeline



- One leakage-free cleaning pipeline feeds every model.
- Every model is evaluated with the same 5-fold Stratified CV — a fair race.

Outline

Dataset overview

Raw sample — ten AQ-10 answers (0/1), age, family history, target

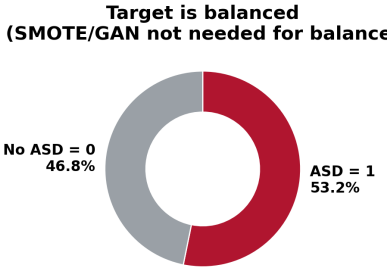
A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	age	sex	fam.	ASD
1	1	1	1	0	0	1	1	0	0	26	f	no	0
1	1	0	1	1	0	1	1	1	1	27	m	yes	1
1	1	0	1	0	0	1	1	0	1	35	f	yes	0
1	0	0	0	0	0	0	1	0	0	40	f	no	0
1	1	1	1	1	0	1	1	1	1	36	m	no	1
1	1	1	1	0	0	0	0	1	0	64	m	no	0

Feature group	Type	Count
AQ-10 answers (`A1`–`A10`)	binary	10
age	numeric	1
sex, jaundice, family ASD	binary	3
ethnicity, relation	categorical	2
`result` (AQ-10 total)	leaky	dropped
country, `used_app_before`	constant	dropped

- Raw: 3,743 rows $\times$ 22 columns, no missing values.
- After cleaning + GAN augmentation: 23,743 rows $\times$ 33 features.
- Target balance: 53% ASD / 47% non-ASD.
- `result` is the sum of `A1`–`A10`, so it is dropped: keeping it would leak the label.

Raw data: quality issues

- 3,743 rows, 22 columns, no missing values.
- age max $= 383 \Rightarrow$ a data-entry error.
- `contry_of_res`, `used_app_before`: one value only (no signal).
- `ethnicity`: duplicates from typos (`White European` vs `White-European`).



The target is balanced (53/47), so SMOTE/GAN are used for robustness, not to fix class imbalance.

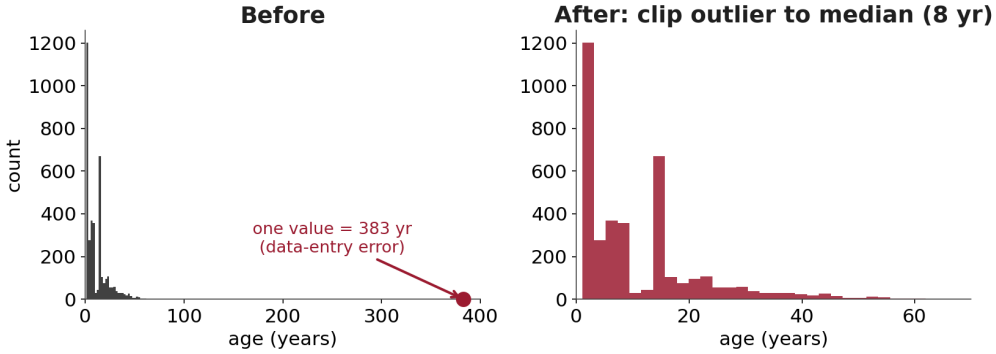
Cleaning pipeline

We run one fixed cleaning script that turns the raw table into model-ready features:

1. Drop columns with no signal (single value, IDs, the leaky total score).
2. Fix the age outlier by clipping it to the median.
3. Merge category typos so the same group is counted once.
4. One-hot encode the remaining text columns.

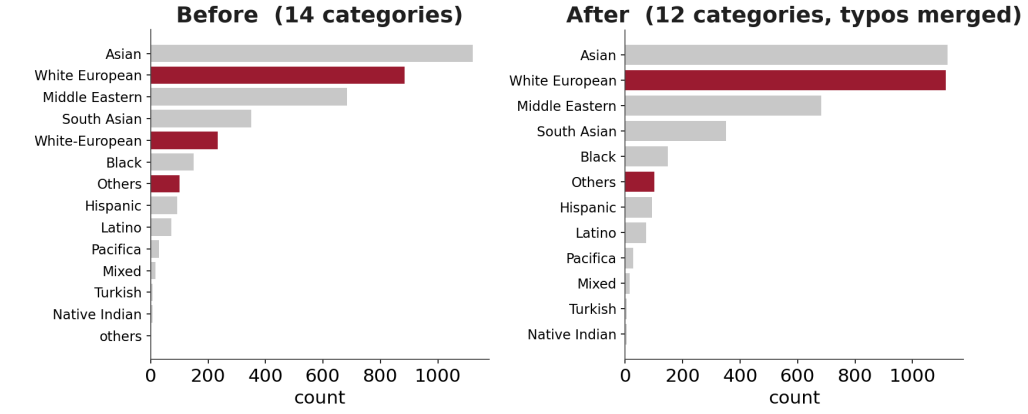
The next three slides show each step on the real data. Output: one clean table reused by all later notebooks.

Step 1 — fix the age outlier



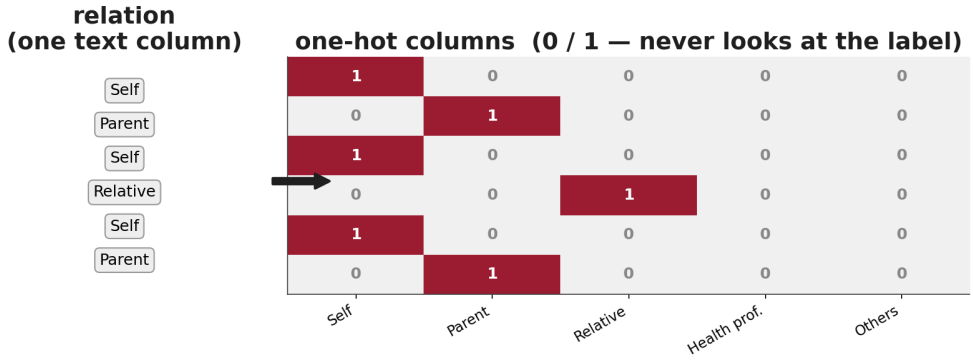
One person is listed as 383 years old. We replace any age above 100 with the median (8 yr), so a single typo cannot distort the model.

Step 2 — merge typo categories



Spelling and dash differences split one real group into two (`White-European` + `White European`). Merging them gives each category its true count.

Step 3 — one-hot encode



Each text category becomes its own 0/1 column. This encoding never looks at the label, so it cannot leak the answer (unlike target encoding).

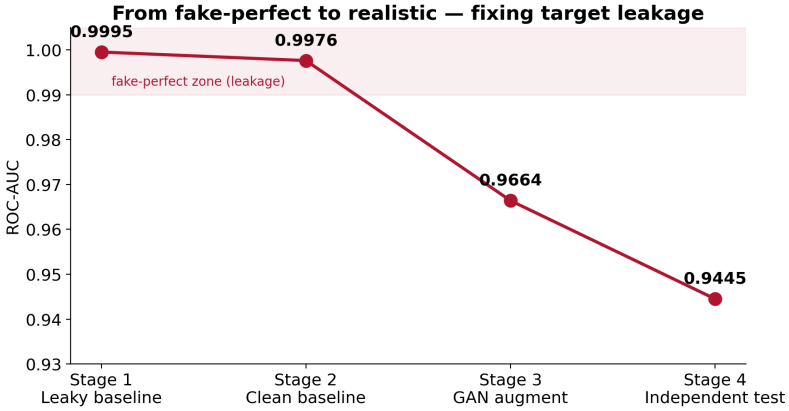
The target-leakage pitfall

An early baseline scored a fake-perfect ROC-AUC $= 0.9995$. Why?

$$\text{enc}(c) = \frac{1}{|\{i : x_i = c\}|} \sum_{i:\, x_i = c} y_i \quad \text{computed on } \textit{all} \text{ data before CV.}$$

- Encoding a category by its mean label hides the answer y inside the feature.
- The model sees the target through that feature $\Rightarrow$ unreal scores.
- Fix: compute encodings inside each fold, or just use one-hot.

Effect of leakage removal and added noise



The score drops as we remove leakage and add realistic noise — the model is becoming trustworthy, not weaker.

Outline

SMOTE — interpolating new samples

Goal: add believable new training rows (not blind copies) so the model is robust.

$$x_{\text{new}} = x_i + \lambda\,(x_i^{\text{nn}} - x_i), \qquad \lambda \sim U(0,1).$$

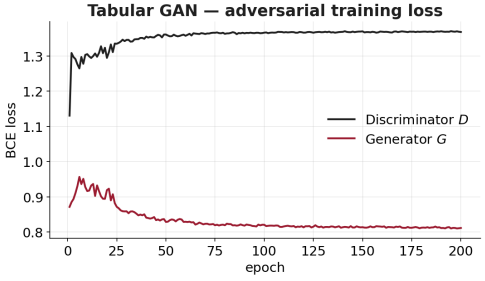
- A new sample is placed on the line between a real point x_i and one of its nearest neighbours x_i^{nn}.
- The new point sits between two real points, so it stays realistic.
- Applied only on the training fold, never on validation $\Rightarrow$ no leakage.

Tabular GAN (PyTorch, from scratch)

Two small networks compete in a min–max game:

$$\min_G \max_D \; \mathbb{E}_x[\log D(x)] + \mathbb{E}_z\big[\log\big(1 - D(G(z))\big)\big] \, .$$

- Generator G maps random noise z to fake patient rows; discriminator D tries to separate real rows from fake ones.
- As D improves, G is pushed to make more realistic rows; the two losses settle near an equilibrium.
- We sampled $+20$k train and $+2$k test rows $\Rightarrow$ a larger, harder, more realistic set.



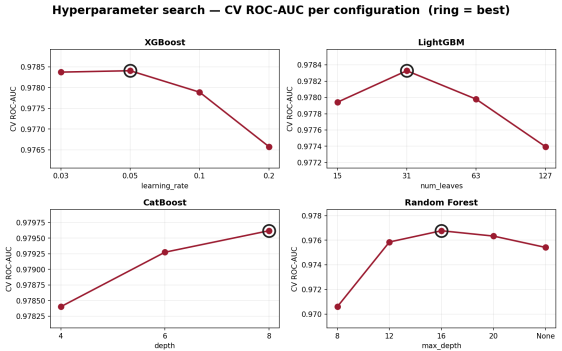
Outline

Evaluation protocol

- One unified 23,743-row cleaned set; the same `StratifiedKFold(5, shuffle, seed=42)` for every model.
- Headline metrics: F1-Score and ROC-AUC (threshold-free); accuracy reported too.
- For each model we search a compact grid of configurations and keep the *best by ROC-AUC* — so every model is compared at its best.
- One leaderboard, extended as each method is added.

Hyperparameter tuning

Each model is tuned with the same 5-fold CV; the best configuration enters the leaderboard.



Validation AUC is flat near the optimum (e.g. XGBoost `lr` 0.03–0.05, LightGBM `num_leaves` 31, CatBoost `depth` 8) — the models are robust, not knife-edge.

Baseline models

- Logistic Regression — linear log-odds: $\log \frac{p}{1-p} = w^\top x + b$. Simple and interpretable.
- Random Forest — average many de-correlated deep trees (bagging) to cut variance.
- AdaBoost — add stumps one by one, re-weighting the samples each one gets wrong.
- XGBoost — regularized gradient boosting; our strongest baseline.

These set the bar that the modern methods must beat.

Leaderboard — baselines

Starting point: XGBoost leads the four baselines.

Model	Acc	F1	ROC-AUC
XGBoost	0.9195	0.9227	0.9784
Random Forest	0.9172	0.9207	0.9768
AdaBoost	0.8668	0.8698	0.9536
Logistic Regression	0.8643	0.8688	0.9473

Gradient boosting — the shared engine

XGBoost, LightGBM, and CatBoost are all additive ensembles of trees built by functional gradient descent:

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \qquad h_m \approx -\frac{\partial L}{\partial F_{m-1}}.$$

- Each new tree h_m fits the negative gradient of the loss L; the learning rate ν shrinks its step.
- XGBoost adds a regularized, second-order objective:

$$\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2,$$

  penalising the number of leaves T and the leaf weights w.
- The three models differ in *how* they grow and regularize these trees.

CatBoost: ordered boosting and target statistics

- Uses oblivious trees (the same split on a whole level), which keeps it fast and well regularized.
- For categories it uses ordered target statistics — a running label average over *earlier* rows only:

$$\hat{x}_i = \frac{\sum_{j<i} \mathbb{1}[x_j = x_i]\, y_j + a\, p}{\sum_{j<i} \mathbb{1}[x_j = x_i] + a}.$$

- Ordered boosting applies the same use-only-the-past rule to the gradients, so a row never sees its own label $\Rightarrow$ no prediction shift.

This directly cures the Phase-1 target-leakage problem.

Leaderboard — + CatBoost

CatBoost takes the top spot.

Model	Acc	F1	ROC-AUC
CatBoost	0.9211	0.9243	0.9796
XGBoost	0.9195	0.9227	0.9784
Random Forest	0.9172	0.9207	0.9768
AdaBoost	0.8668	0.8698	0.9536
Logistic Regression	0.8643	0.8688	0.9473

TabM: parameter-efficient deep ensembling (BatchEnsemble)

- A plain MLP is one network. An ensemble of k MLPs predicts better, but costs about $k\times$ to train.
- TabM keeps one shared weight matrix W and gives each member a cheap rank-1 adapter (r_k, s_k), so the k members cost $\approx 1\times$:

$$W_k = W \odot (r_k\, s_k^\top), \qquad \hat{p} = \frac{1}{k}\sum_{m=1}^{k} \mathrm{softmax}\big(f_{W_m}(x)\big).$$

- Each member sees the data a little differently, so their mistakes differ.
- Averaging the members cancels noise and lowers variance, giving better-calibrated probabilities.

Leaderboard — + TabM

TabM edges ahead of CatBoost on ROC-AUC (0.9798).

Model	Acc	F1	ROC-AUC
TabM	0.9207	0.9246	0.9798
CatBoost	0.9211	0.9243	0.9796
XGBoost	0.9195	0.9227	0.9784
Random Forest	0.9172	0.9207	0.9768
AdaBoost	0.8668	0.8698	0.9536
Logistic Regression	0.8643	0.8688	0.9473

LightGBM: histogram and leaf-wise boosting

- Same gradient-boosting objective as XGBoost, but tuned for speed.
- Bins each continuous feature into a histogram (e.g. 255 buckets), so the best split is found by scanning buckets, not every value.
- Grows trees leaf-wise (always split the leaf that cuts the loss most), reaching lower error with fewer splits.
- GOSS and exclusive feature bundling further cut the work per tree without changing the loss it fits.

Completes the gradient-boosting trio.

Leaderboard — + LightGBM

LightGBM lands just behind XGBoost.

Model	Acc	F1	ROC-AUC
TabM	0.9207	0.9246	0.9798
CatBoost	0.9211	0.9243	0.9796
XGBoost	0.9195	0.9227	0.9784
LightGBM	0.9185	0.9218	0.9783
Random Forest	0.9172	0.9207	0.9768
AdaBoost	0.8668	0.8698	0.9536
Logistic Regression	0.8643	0.8688	0.9473

TabPFN: in-context learning for tables

- A transformer trained *once* on millions of synthetic tables, so it learns a general prior over how tabular data behaves.
- At inference there is no training step: the entire training set and a test row are supplied as context, and the label is predicted in a single forward pass (in-context learning):

$$p\big(y \mid x, \mathcal{D}_{\text{train}}\big) \;\approx\; \text{one in-context forward pass.}$$

- Because it learned a prior over datasets, it approximates the Bayesian posterior predictive — like averaging over many plausible models.

Best single model out of the box (F1 0.9275, AUC 0.9810); built for small/medium tables.

Leaderboard — + TabPFN

TabPFN leads with no tuning at all.

Model	Acc	F1	ROC-AUC
TabPFN	0.9241	0.9275	0.9810
TabM	0.9207	0.9246	0.9798
CatBoost	0.9211	0.9243	0.9796
XGBoost	0.9195	0.9227	0.9784
LightGBM	0.9185	0.9218	0.9783
Random Forest	0.9172	0.9207	0.9768
AdaBoost	0.8668	0.8698	0.9536
Logistic Regression	0.8643	0.8688	0.9473

Stacking and blending

- Train several diverse base models with CV and keep their out-of-fold (OOF) predictions — each row is predicted only by models that did not see it, so it stays leakage-free.
- Blending — average the probabilities: $\hat{p} = \frac{1}{M} \sum_m p_m(x)$.
- Stacking — a logistic-regression meta-learner reads those predictions and learns how much to trust each model:

$$\hat{p} = \sigma\Big(\sum_m w_m \, p_m(x) + b \Big).$$

- The base models make different mistakes, so combining them cancels errors and is steadier than any single model.

Base learners: TabPFN + CatBoost + LightGBM + XGBoost.

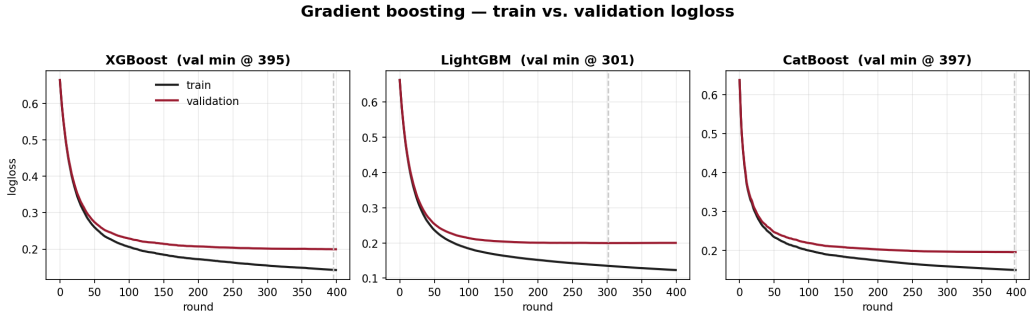
Leaderboard — + stacking and blending

The ensembles sit just below TabPFN.

Model	Acc	F1	ROC-AUC
TabPFN	0.9241	0.9275	0.9810
Stacking (LR)	0.9232	0.9266	0.9807
Blend (avg 4)	0.9226	0.9258	0.9799
TabM	0.9207	0.9246	0.9798
CatBoost	0.9211	0.9243	0.9796
XGBoost	0.9195	0.9227	0.9784
LightGBM	0.9185	0.9218	0.9783
Random Forest	0.9172	0.9207	0.9768
AdaBoost	0.8668	0.8698	0.9536
Logistic Regression	0.8643	0.8688	0.9473

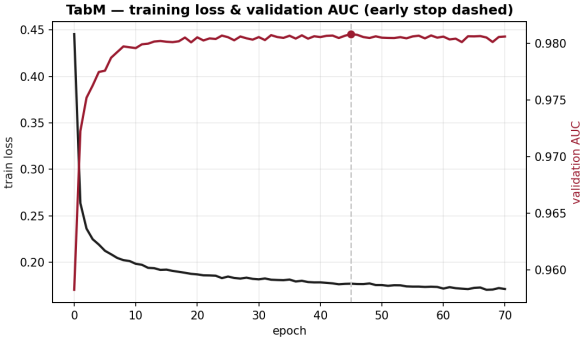
Outline

Training diagnostics — gradient boosting



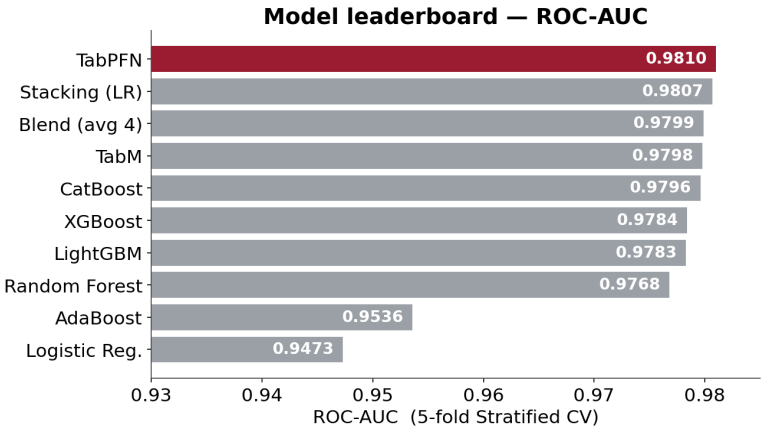
Train logloss keeps falling while validation logloss flattens and then creeps up — the gap is the onset of overfitting, and the marked round is where we would stop. All three converge to a similar validation loss.

Training diagnostics — TabM



- Training cross-entropy (black) falls smoothly as the ensemble fits.
- Validation AUC (red) rises, then plateaus; we keep the best-AUC epoch (dashed) and stop early.
- No runaway gap $\Rightarrow$ the rank-1 ensemble regularizes the MLP.

Final leaderboard

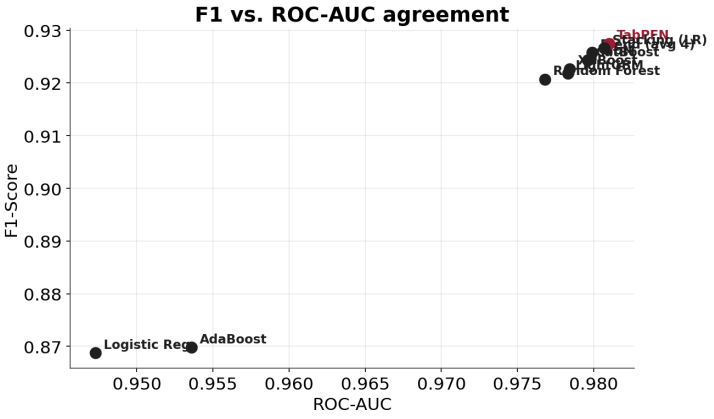


From LogReg 0.947 to TabPFN 0.981 ROC-AUC. The top of the table is a foundation model plus ensembles; a zero-tuning model wins.

Why each model wins or lags

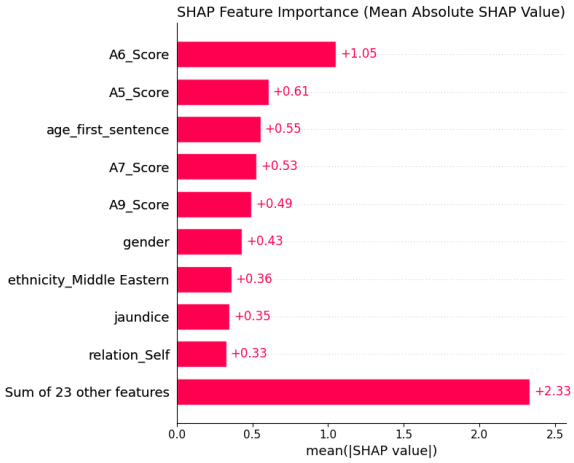
Model	Why it works	Why it lags
LogReg, AdaBoost	simple, interpretable	linear / shallow stumps underfit the AQ-10 $\times$ demographic interactions
Random Forest	low bias, captures interactions	higher variance than regularized boosting
XGBoost / LightGBM / CatBoost	regularized boosting models interactions; CatBoost's ordered TS is leakage-safe	gains over each other are small; need tuning
TabM	deep ensemble lowers variance, well-calibrated	a more complex deep model for only a marginal gain over boosting
TabPFN	learned Bayesian prior over tables; best, zero tuning	assumes small/medium tables; heavier at inference
Stacking / Blend	decorrelated errors cancel $\Rightarrow$ steadiest	only as strong as its base models

F1 vs. ROC-AUC agreement



F1 and ROC-AUC rank the top models in the same order, so the ranking is not an artefact of one metric. The gaps near the top are small — the strong methods are close.

Explainable AI — global feature importance

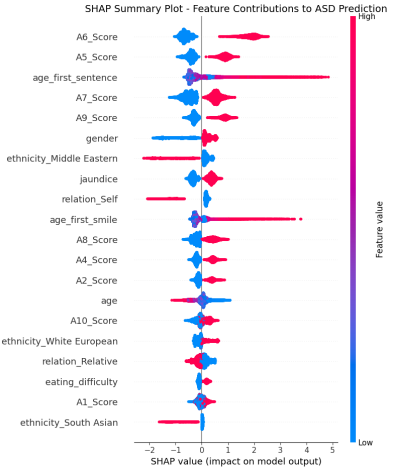


A SHAP value is a feature's fair contribution (a Shapley value):

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F|-|S|-1)!}{|F|!} \left(f(S \cup \{i\}) - f(S)\right)$$

- AQ-10 items (A6, A5, A7, A9) sit on top.
- Developmental milestones (`age_first_sentence`) matter just as much.

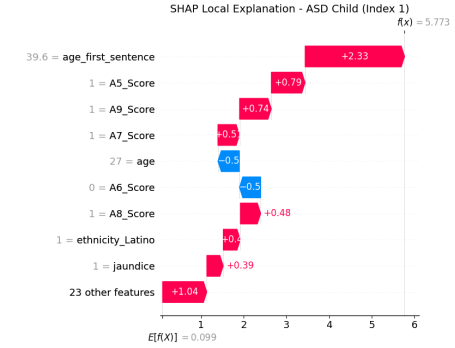
SHAP — how each feature pushes the output



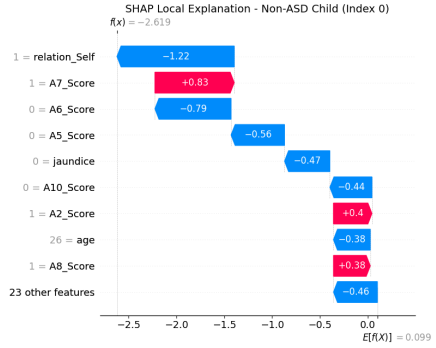
- One dot = one patient; colour = feature value (red high, blue low).
- Right of 0 ⇒ pushes towards ASD.
- High AQ-10 scores (red, on the right) raise ASD risk — clinically sensible.
- A late `age_first_sentence` (red) strongly raises risk.

SHAP — per-patient explanations

ASD case $f(x) = 5.77$



Non-ASD case $f(x) = -2.62$



Red bars push up (towards ASD), blue bars push down. Same model — a transparent reason for every individual prediction.

Outline

Conclusion

1. Built a leakage-free pipeline — honest scores over fake-perfect ones.
2. GAN/SMOTE augmentation made the model robust to realistic noise.
3. A fair race of 10 tuned models: TabPFN wins; stacking is a steady runner-up.
4. SHAP confirms clinically sensible drivers.

References

- Kaggle, *Autism Prediction Challenge* dataset.
- Chawla et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. JAIR.
- Goodfellow et al. (2014). *Generative Adversarial Networks*. arXiv:1406.2661.
- Chen & Guestrin (2016). *XGBoost: A Scalable Tree Boosting System*. arXiv:1603.02754.
- Ke et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS.
- Prokhorenkova et al. (2018). *CatBoost: Unbiased Boosting with Categorical Features*. arXiv:1706.09516.

References (cont.)

- Lundberg & Lee (2017). *A Unified Approach to Interpreting Model Predictions (SHAP)*. arXiv:1705.07874.
- Hollmann et al. (2022). *TabPFN: A Transformer That Solves Small Tabular Problems in a Second*. arXiv:2207.01848.
- Gorishniy et al. (2024). *TabM: Advancing Tabular DL with Parameter-Efficient Ensembling*. arXiv:2410.24210.

THANK YOU!

Questions & discussion